

Secure Login System

Document challenges faced and solutions implemented

Day 1 – Project Setup

- Challenges
 - Picking the right file structure that scales.
 - Ensuring VS Code plugins don't conflict or slow your machine.
 - Forgetting .gitignore and accidentally leaking environment configs.
- Solutions
 - Use a modular structure early: frontend/, backend/, configs/, docs/, tests/.
 - Install only essentials: Python/Node extensions, Prettier, GitLens.
 - Add a .gitignore immediately (node_modules, __pycache__, .env, venv).
 - Commit from day zero—version control is your black box recorder.

Day 2 – Frontend Development

- Challenges
 - Getting pixel-perfect designs without bloated CSS.
 - Form validation logic becoming spaghetti
 - User role dropdown not reflecting backend requirements.
- Solutions
 - Keep UI atomic—small reusable components.
 - Use clean regex-based validation: email, password strength, required fields.
 - Mirror backend's role schema (Admin, User) directly in the form.
 - Test structure early using fake data to avoid surprises later.

Day 3 – Backend Setup

- Challenges
 - Deciding between Flask and Node.js—architecture drift.
 - Database connection failures (network, URI issues).
 - Circular imports when structuring routes/models.
- Solutions
 - Pick one stack and commit. Node.js pairs perfectly with MongoDB for quick iteration.
 - Store DB URI in .env and test with a simple connection script first.
 - Use a layered architecture: routes/, controllers/, models/, so nothing loops.
 - Run a quick health-check endpoint to ensure the server is behaving.

Day 4 – User Registration

- Challenges
 - Password hashing errors (bcrypt version mismatch or async issues).
 - Duplicate email/username logic failing silently.
 - Validation mismatch between frontend & backend.
- Solutions
 - Use bcrypt with salt rounds between 10–12.
 - Add unique indexes in Mongo/MySQL for email and username.
 - Build backend validation separately—never trust the client.
 - Write a small seed script to test registrations programmatically.

Day 5 – User Login

- Challenges
 - JWT signing/verification mistakes.
 - Incorrect password comparison due to hashing misunderstandings.
 - Sessions expiring too quickly or never expiring.
- Solutions
 - Build a clean auth service: `generateToken()`, `verifyToken`
 - Always compare hashed passwords using bcrypt's dedicated compare method
 - Use refresh tokens if needed, but keep access tokens short-lived (15–30 mins).
 - Log failed login attempts for Day 7's logout feature.

Day 6 – Role-Based Access Control

- Challenges
 - Middleware accidentally lets normal users into admin routes.
 - Dashboard not syncing with backend permissions.
 - Admin actions affecting system integrity (e.g., admin deleting admin).
- Solutions
 - Create a strict RBAC middleware that checks: token → role → permission.
 - Protect admin actions behind verified role checks AND route-level guards.
 - Implement soft-delete or audit logs for admin actions.
 - Keep the admin dashboard modular so new features don't break access logic.

Day 7 – Security Enhancements

- Challenges
 - Developers adding "temporary" debug routes and forgetting to remove them
 - CAPTCHA integration breaking UI/UX
 - Account lockout being too strict or too lax
- Solutions
 - Audit routes and endpoints—purge anything not production-ready.
 - Use Google reCAPTCHA or hCaptcha with graceful fallback.
 - Build a lockout strategy:
 - 5 failed attempts → 15-minute lock
 - Keep timestamps of attempts
 - Sanitize all inputs using backend validators regardless of DB type.

Day 8 – Testing & Debugging

- Challenges
 - Edge cases slipping through (email casing, whitespace, weird symbols).
 - Race conditions in signup/login flow.
 - Testing only "happy paths."
- Solutions
 - Create a dedicated tests/ folder with automated tests (Jest/PyTest).
 - Test negative scenarios intentionally: wrong credentials, empty fields, duplicate emails.
 - Document issues like a post-incident SOC report—root cause → fix → prevention.
 - Run end-to-end workflow tests: register → login → access dashboard → logout.